Hierarchical k-GNN Explanations via Generative Interpretation Networks

Comparing 1-GNN and 1-2-GNN for Model-Level Graph Classification Interpretability

[Author Name]

March 2026

Abstract

Graph Neural Networks (GNNs) are powerful tools for graph classification, yet their decision processes remain opaque. Higher-order k-GNNs, which operate on k-element subsets of nodes, provably increase expressive power beyond the Weisfeiler–Leman hierarchy, but it is unclear whether this additional expressiveness leads the model to learn qualitatively different internal representations. We investigate this question by comparing a standard 1-GNN with a hierarchical 1-2-GNN using the GIN-Graph framework for model-level explanation generation. Our pipeline trains k-GNN classifiers on the MUTAG and PROTEINS benchmarks, then uses a Wasserstein GAN with gradient penalty to generate class-representative explanation graphs guided by the pretrained classifier. We introduce a fully differentiable dense wrapper that maintains gradient flow through both node features and adjacency matrices for hierarchical models. On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate different explanations: the 1-GNN associates mutagenicity with nitrogen-containing structures while the 1-2-GNN favours halogenated compounds, indicating that the two architectures encode different structural features of the same class. Cross-model classification of 500 generated graphs per class confirms this divergence quantitatively. On PROTEINS, 1-GNN-generated Enzyme graphs are rejected by the 1-2-GNN (9.4% agreement), while 1-2-GNN-generated graphs are accepted by both models (98–100% agreement), indicating that the 1-2-GNN learns a more selective decision boundary in its pairwise feature space. These results demonstrate that different GNN architectures develop qualitatively different internal representations of graph classes, and that model-level explanations can reveal these differences even when classification accuracy is comparable.

Contents

1 Introduction

Graph Neural Networks (GNNs) have become the standard approach for learning on graph-structured data, achieving strong results on tasks ranging from molecular property prediction to social network analysis. Despite their effectiveness, GNNs suffer from the same interpretability challenges as other deep learning models: their predictions are difficult to explain in human-understandable terms.

The expressiveness of standard message-passing GNNs is bounded by the 1-dimensional Weisfeiler–Leman (1-WL) graph isomorphism test [Morris et al.(2019)]. Morris et al. [Morris et al.(2019)] proposed k-GNNs that operate on k-element subsets of nodes, achieving expressiveness equivalent to the k-WL test. In theory, higher-order k-GNNs can distinguish graph structures that standard GNNs cannot.

A natural question arises: *does this additional structural expressiveness lead the model to learn different internal representations of graph classes, and can we observe these differences through generated explanations?* If a model captures richer structural patterns, the explanations it produces (representative graphs that characterise each class) may reveal what structural features each architecture has learned to associate with each class.

We investigate this question using the GIN-Graph framework [Sun et al.(2025)], which generates model-level explanation graphs through a GAN-based approach guided by a pretrained classifier. Model-level explanations aim to answer the question "what does a typical graph of class c look like according to the model?" by generating new graphs that maximise the model's confidence for that class while remaining structurally realistic. This contrasts with instance-level methods that explain individual predictions by identifying important subgraphs.

Specifically, we compare:

- A **1-GNN**: standard node-level message passing, equivalent to 1-WL;
- A **1-2-GNN**: hierarchical architecture combining node-level (1-GNN) and pairwise (2-GNN) message passing, achieving expressiveness beyond 1-WL.

This work:

1. Reuses the dense 1-GNN wrapper from GIN-Graph [Sun et al.(2025)] and applies the same logic to the 2-GNN branch of a 1-2-GNN classifier, so that GIN-Graph training can be run against a hierarchical k-GNN.
2. Computes the embedding similarity component s of the validation score via cosine similarity between generated and real graph embeddings. The GIN-Graph reference implementation uses the prediction probability p as a proxy for s; we use the actual embedding.
3. Compares the explanations produced by the two architectures on MUTAG and PROTEINS. The two models produce different explanation structures and show asymmetric cross-model classification agreement, with some configurations showing near-zero agreement.

2 Background

2.1 Graph Neural Networks

A graph $G = (V, E)$ consists of a node set V with $|V| = n$ and an edge set $E \subseteq V \times V$. Each node $v \in V$ carries a feature vector $\mathbf{x}_v \in \mathbb{R}^d$. We denote the adjacency matrix as $\mathbf{A} \in \{0,1\}^{n \times n}$ where $A_{uv} = 1$ if $(u, v) \in E$, and the feature matrix as $\mathbf{X} \in \mathbb{R}^{n \times d}$ where row v is $\mathbf{x}_v$.

GNNs learn node representations through iterative *message passing*. At each layer t, a node updates its representation by aggregating information from its neighbours:

$$\mathbf{h}_v^{(t)} = \mathrm{UPDATE}\big(\mathbf{h}_v^{(t-1)},\ \mathrm{AGGREGATE}\big(\{\!\{\mathbf{h}_u^{(t-1)} : u \in \mathcal{N}(v)\}\!\}\big)\big)\,, \tag{1}$$

where $\mathcal{N}(v) = \{u \in V : (u, v) \in E\}$ denotes the neighbourhood of v, $\mathbf{h}_v^{(0)} = \mathbf{x}_v$ is the initial node feature, and $\{\!\{\cdot\}\!\}$ denotes a multiset.

The choice of UPDATE and AGGREGATE functions determines the specific GNN variant. In our implementation, these are parameterised by separate linear transformations for the self-feature and the aggregated neighbour features. After T layers of message passing, a *readout* function pools all node embeddings into a single graph-level representation:

$$\mathbf{h}_G = \mathrm{READOUT}\big(\{\!\{\mathbf{h}_v^{(T)} : v \in V\}\!\}\big)\,. \tag{2}$$

Common choices for READOUT include sum, mean, and max pooling over the node set. We use sum pooling for the classifier readouts, as it preserves information about both the features and the number of nodes.

2.2 The k-WL Hierarchy and k-GNNs

The 1-dimensional *Weisfeiler–Leman* (1-WL) test decides graph isomorphism up to a known failure class by iteratively hashing node labels. Each node's new label is a hash of its current label together with the multiset of its neighbours' labels. After convergence, two graphs are declared possibly isomorphic iff their label histograms agree.

The standard failure mode is that 1-WL cannot tell apart two graphs in which every node has the same local signature. The canonical example is two disjoint triangles versus a single hexagon: every node is degree 2 with two degree-2 neighbours, so the histograms match at every iteration even though the graphs are not isomorphic.

The k-WL test fixes this by hashing over k-tuples of nodes rather than individual nodes, which makes non-local structural differences visible; 2-WL already distinguishes the triangles-vs-hexagon example. Morris et al. [Morris et al.(2019)] showed that 1-GNNs are at most as powerful as 1-WL in distinguishing power, and that k-GNNs operating on k-element node subsets ("k-sets") are at most as powerful as k-WL (their Propositions 1–4). The Cai–Fürer–Immerman construction [Cai et al.(1992)] further shows $(k{+}1)$-WL is strictly more powerful than k-WL for all $k \geq 2$. This is the motivation for considering a 1-2-GNN: it augments node-level message passing with pair-level message passing and can therefore capture structural distinctions invisible to a standard 1-GNN. We rely on these results as background and do not reproduce them here.

2.3 GIN-Graph

GIN-Graph [Sun et al.(2025)] is a model-level explanation method that generates class-representative graphs using a generative adversarial approach. The key idea is to train a generator $\mathcal{G}$ to produce graphs that satisfy two objectives simultaneously:

1. **Realism**: generated graphs should be structurally similar to real graphs from the dataset, enforced by a WGAN-GP discriminator.
2. **Class specificity**: generated graphs should be confidently classified as the target class by the pretrained GNN, enforced by a classification guidance loss.

The generator maps noise $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ to a graph $(\tilde{A}, \tilde{X})$ using Gumbel-Softmax for differentiable discrete sampling. The training objective balances the two losses through a dynamic weighting schedule that shifts focus from realism (early training) to class specificity (late training).

3 Method

This section details the mathematical formulations of our two classifier architectures (1-GNN and 1-2-GNN), the GIN-Graph generator, the training procedure, and the differentiable dense wrapper that bridges them.

3.1 1-GNN Architecture

Our 1-GNN implements message passing with separate transformations for self-features and neighbour-aggregated features. At layer t, the update rule for node v is:

$$\mathbf{h}_v^{(t)} = \sigma\left(\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)} + \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}\right), \tag{3}$$

where $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)} \in \mathbb{R}^{d_{t-1} \times d_t}$ are learnable weight matrices, σ is the ReLU activation function $\sigma(x) = \max(0, x)$, and $d_0 = d$ (the input feature dimension). The self-transformation $\mathbf{h}_v^{(t-1)}\mathbf{W}_1^{(t)}$ allows the model to retain and transform the node's own representation, while the neighbour term $\sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(t-1)}\mathbf{W}_2^{(t)}$ aggregates structural context.

This can equivalently be written in matrix form for the entire graph:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{4}$$

where $\mathbf{H}^{(t)} \in \mathbb{R}^{n \times d_t}$ is the matrix of all node representations at layer t, and $\mathbf{A} \in \{0, 1\}^{n \times n}$ is the adjacency matrix. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ computes the sum of neighbour features for every node simultaneously.

After $T = 3$ layers with hidden dimension $d_1 = d_2 = d_3 = 64$, the graph embedding is obtained via sum pooling:

$$\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T)} \in \mathbb{R}^{d_T}. \tag{5}$$

This embedding is fed through a two-layer classifier MLP with ReLU activation and dropout:

$$\hat{y} = \mathrm{softmax}\Big(\mathbf{W}_o\,\sigma(\mathbf{W}_c\,\mathbf{h}_G^{(1)})\Big)\,, \tag{6}$$

where $\mathbf{W}_c \in \mathbb{R}^{d_T \times d_T}$ and $\mathbf{W}_o \in \mathbb{R}^{d_T \times C}$ with C being the number of classes.

3.2 2-GNN and the Hierarchical 1-2-GNN

A 1-GNN aggregates only over individual node neighbourhoods, and two graphs in which every node has the same local signature are therefore indistinguishable to it. The canonical instance is two disjoint triangles compared with a single hexagon: every node has degree two with two degree-two neighbours, so the aggregated node features coincide in both graphs. Representing the graph at the level of node pairs restores the missing structural signal, because a pair carries information about whether its two elements are adjacent and about the neighbourhoods of both elements simultaneously.

A pair feature is a fixed-dimensional vector attached to each unordered node pair that summarises the pair together with an indicator of its edge status in the original graph. For each pair $s = \{u, v\}$ with canonical ordering $u < v$, the initial pair feature is built from the two 1-GNN node embeddings together with the edge indicator:

$$\mathbf{f}_s^{(0)} = \big[\mathbf{h}_u^{(T)} \parallel \mathbf{h}_v^{(T)} \parallel \mathbb{1}[(u, v) \in E]\big] \in \mathbb{R}^{2d_T+1}, \tag{7}$$

The concatenation of $\mathbf{h}_u^{(T)}$ and $\mathbf{h}_v^{(T)}$ carries the node-level context from the 1-GNN, and the bond bit $\mathbb{1}[(u, v) \in E] \in \{0, 1\}$ acts as the isomorphism-type marker for the pair, distinguishing bonded pairs from unbonded pairs that would otherwise share the same node-level content.

Message passing on pairs uses a neighbourhood in which each neighbour of $s = \{u, v\}$ is obtained by replacing one of its two elements with a node adjacent in G to the removed element:

$$\mathcal{N}_L(\{u, v\}) = \{\{v, w\} : w \neq u,\ (u, w) \in E\} \cup \{\{u, w\} : w \neq v,\ (v, w) \in E\}\,. \tag{8}$$

The first set replaces u by a node w adjacent to u in G; the second replaces v by a node w adjacent to v in G. The neighbourhood relation among pairs therefore inherits its structure from the edge structure of the underlying graph.

The pair-level update combines the pair's own features with an aggregation over its pair-neighbourhood through two weight matrices:

$$\mathbf{f}_s^{(\ell+1)} = \sigma\left(\mathbf{f}_s^{(\ell)}\mathbf{W}_3^{(\ell)} + \sum_{s' \in \mathcal{N}_L(s)} \mathbf{f}_{s'}^{(\ell)}\mathbf{W}_4^{(\ell)}\right), \tag{9}$$

The self-transformation $\mathbf{W}_3^{(\ell)}$ preserves the pair's own representation across layers, and the neighbour-transformation $\mathbf{W}_4^{(\ell)}$ propagates information from adjacent pairs. The 2-GNN uses $T_2 = 2$ such layers.

The hierarchical 1-2-GNN runs the 1-GNN first for $T_1 = 3$ layers to obtain node embeddings, uses these embeddings to form the initial pair features via Equation (7), and then

runs T_2 layers of pair-level message passing. The graph representation is the concatenation of a node-level and a pair-level readout:

$$\mathbf{h}_G = \left[\mathbf{h}_G^{(1)} \parallel \mathbf{h}_G^{(2)}\right] \in \mathbb{R}^{2d_T}, \tag{10}$$

The node-level readout sums the final node embeddings, $\mathbf{h}_G^{(1)} = \sum_{v \in V} \mathbf{h}_v^{(T_1)}$, and the pair-level readout sums the final pair features:

$$\mathbf{h}_G^{(2)} = \sum_{\{u,v\} \in \binom{V}{2}} \mathbf{f}_{\{u,v\}}^{(T_2)} \in \mathbb{R}^{d_T}. \tag{11}$$

The concatenated embedding $\mathbf{h}_G$ is passed through the classifier of Equation (6) with input dimension $2d_T$ in place of d_T.

Scalability. The number of pairs grows as $\binom{n}{2} = n(n-1)/2$, which becomes prohibitive for large graphs. For PROTEINS (up to 620 nodes, yielding up to 191,890 pairs), all pairs are processed using a numba-accelerated sparse kernel with a CSR adjacency representation, giving $O(\text{degree})$ neighbour lookup instead of the $O(N)$ cost of dense masks. The full pairwise structure is therefore computed exactly, without sampling.

3.3 GIN-Graph Generator

MLP backbone. The generator $\mathcal{G}$ maps a latent noise vector to a graph through a shared two-layer MLP backbone with two output heads, one producing adjacency logits and one producing node-feature logits:

$$\mathbf{h} = \text{LeakyReLU}(\mathbf{W}_2 \, \text{LeakyReLU}(\mathbf{W}_1 \mathbf{z})) \in \mathbb{R}^{2d_h}, \tag{12}$$

$$\tilde{A}_{\text{raw}} = \text{sym}(\text{reshape}(\mathbf{h}\,\mathbf{W}_A,\ N \times N)), \tag{13}$$

$$\tilde{X}_{\text{raw}} = \text{reshape}(\mathbf{h}\,\mathbf{W}_X,\ N \times D). \tag{14}$$

The latent vector $\mathbf{z} \in \mathbb{R}^{d_z}$ is drawn from a standard normal, with $d_z = 32$ and hidden width $d_h = 128$. The adjacency head produces an $N \times N$ matrix, where N is the maximum number of nodes in the generated graph, and the feature head produces an $N \times D$ matrix, where D is the number of node feature types in the target dataset. The activation is $\text{LeakyReLU}_\alpha(x) = \max(\alpha x, x)$ with negative slope $\alpha = 0.2$; the nonzero slope on the negative side keeps gradients flowing through units with negative pre-activations, which avoids collapsing diversity of the generator output when a subset of hidden units would otherwise become dead under ReLU. The symmetrisation $\text{sym}(\mathbf{M}) = \frac{1}{2}(\mathbf{M} + \mathbf{M}^\top)$ enforces undirected adjacency. The outputs of Equations (13)–(14) are real-valued logits, not yet a graph; a discrete graph is obtained in the following step.

Gumbel-Softmax sampling. The quantities the generator must produce are discrete: each edge is binary, and each node type is a choice among D categories. Selecting these by arg max or by thresholding would make the mapping from logits to outputs piecewise constant, with zero gradient almost everywhere. The Gumbel-Softmax trick [Jang et al.(2017)] replaces this hard sampling step with a differentiable relaxation. For unnormalised log-probabilities $\pi_1, \ldots, \pi_K$, a Gumbel-Softmax sample is defined as

$$y_i = \frac{\exp((\pi_i + g_i)/\tau)}{\sum_{j=1}^{K} \exp((\pi_j + g_j)/\tau)}, \quad g_i = -\log(-\log(u_i)), \quad u_i \sim \text{Uniform}(0,1). \tag{15}$$

The Gumbel samples g_i carry all of the stochasticity in this expression; the logits π_i enter only through deterministic operations. Gradients with respect to π_i therefore propagate through Equation (15). The temperature $\tau > 0$ controls sharpness: as $\tau \to 0$ the softmax approaches a one-hot vector, and as $\tau \to \infty$ it approaches the uniform distribution.

Straight-through estimator. The classifier used to guide the generator was trained on discrete graphs with $\{0, 1\}$ adjacency and one-hot node features, and it is not meaningful to feed it continuous Gumbel-Softmax outputs at training time. The straight-through estimator resolves this by combining a hard forward pass with a soft backward pass: the hard one-hot sample $\tilde{y}_{\text{hard}} = \text{onehot}(\arg\max_i y_i)$ is used in the forward computation, while the gradient is taken with respect to the continuous sample $\tilde{y}_{\text{soft}}$ from Equation (15). The standard implementation uses the identity

$$\tilde{y} = \tilde{y}_{\text{hard}} + \tilde{y}_{\text{soft}} - \text{stopgrad}(\tilde{y}_{\text{soft}}),$$

where $\text{stopgrad}(\cdot)$ instructs autograd to treat its argument as a constant, so it contributes zero to any derivative. The value of $\tilde{y}$ is simply $\tilde{y}_{\text{hard}}$, since $\tilde{y}_{\text{soft}}$ and $\text{stopgrad}(\tilde{y}_{\text{soft}})$ take the same numerical value and cancel. Differentiating with respect to the generator parameters θ,

$$\frac{\partial \tilde{y}}{\partial \theta} = \underbrace{\frac{\partial \tilde{y}_{\text{hard}}}{\partial \theta}}_{=0} + \frac{\partial \tilde{y}_{\text{soft}}}{\partial \theta} - \underbrace{\frac{\partial\, \text{stopgrad}(\tilde{y}_{\text{soft}})}{\partial \theta}}_{=0} = \frac{\partial \tilde{y}_{\text{soft}}}{\partial \theta}.$$

The $\arg\max$ in $\tilde{y}_{\text{hard}}$ has zero gradient almost everywhere, and the stop-gradient term contributes zero by construction, so only the soft Gumbel-Softmax sample passes gradient to θ. The classifier therefore sees a discrete graph on the forward pass and the generator receives the Gumbel-Softmax gradient on the backward pass.

Application to the two heads. Each potential edge (i, j) is a binary choice between edge and no-edge, so 2-class logits $[\tilde{A}_{\text{raw}}[i, j],\ -\tilde{A}_{\text{raw}}[i, j]]$ are formed and passed through the Gumbel-Softmax with $K = 2$. Symmetry of the output adjacency is enforced by sampling only the upper triangle and mirroring it to the lower triangle:

$$\tilde{A}_{\text{upper}} = \text{triu}(\text{GumbelSoftmax}(\tilde{A}_{\text{raw}}, \tau)), \quad \tilde{A} = \tilde{A}_{\text{upper}} + \tilde{A}_{\text{upper}}^{\top}. \tag{16}$$

For node features, each node selects one of D types (for example, seven atom types on MUTAG), so the Gumbel-Softmax is applied with $K = D$ per node to obtain one-hot feature rows:

$$\tilde{X}[v, :] = \text{GumbelSoftmax}(\tilde{X}_{\text{raw}}[v, :],\ \tau) \in \{0, 1\}^D. \tag{17}$$

The temperature is $\tau = 1.0$ during training, which leaves the relaxation smooth enough to give useful gradients, and $\tau = 0.1$ during evaluation, which produces sharper, effectively discrete outputs.

3.4 Training Objective

The total generator loss combines adversarial, GNN-guidance, and structural regularisation terms:

$$\mathcal{L}_G = (1 - \lambda(t))\, \mathcal{L}_{\text{GAN}} + \lambda(t)\, \mathcal{L}_{\text{GNN}} + \mathcal{L}_{\text{reg}}, \tag{18}$$

where $\lambda(t) \in [0, 1]$ is a dynamic weight that increases over training, and $\mathcal{L}_{\text{reg}}$ comprises structural regularisation losses described below. The three terms correspond to three

goals: making the generated graphs look realistic ($\mathcal{L}_{\text{GAN}}$), making them belong to the target class ($\mathcal{L}_{\text{GNN}}$), and making them the right size and composition ($\mathcal{L}_{\text{reg}}$). We now explain each in turn, beginning with the adversarial component and the discriminator that drives it.

3.4.1 $\mathcal{L}_{\text{GAN}}$: The Discriminator

The discriminator $\mathcal{D}$ is a graph neural network that maps a graph to a single scalar. It operates on dense batched inputs (node features $\mathbf{X} \in \mathbb{R}^{B\times N\times D}$ and adjacency $\mathbf{A} \in [0,1]^{B\times N\times N}$) so that the same forward pass handles both real and generated graphs. The architecture has three stages:

1. **Two dense GCN layers** with hidden dimension 128 and ReLU activations. Each layer computes $\mathbf{H}^{(t)} = \sigma(\hat{\mathbf{A}}\,\mathbf{H}^{(t-1)}\,\mathbf{W}^{(t)})$, where $\hat{\mathbf{A}}$ is the adjacency with self-loops added and symmetric degree normalisation applied.
2. **Mean pooling** across the node dimension: $\mathbf{h}_G = \frac{1}{N}\sum_{v=1}^{N} \mathbf{h}_v^{(2)} \in \mathbb{R}^{128}$. This produces a single graph-level vector per graph.
3. **Two-layer MLP** with LeakyReLU activation (negative slope $\alpha = 0.2$, as in Section 3.3): $\mathcal{D}(G) = \mathbf{w}^\top \text{LeakyReLU}_{0.2}(\mathbf{W}\,\mathbf{h}_G) \in \mathbb{R}$. In the discriminator the LeakyReLU choice is standard for GAN training, where ReLU's zero-gradient half-plane can cause dead units that stop contributing signal to the generator during adversarial updates.

Higher values of $\mathcal{D}(G)$ indicate that the graph appears real; lower values indicate that it appears generated. There is no sigmoid or softmax at the output, so $\mathcal{D}(G)$ is not a probability. In a standard binary classifier, the scalar would pass through a sigmoid $\sigma(x) = 1/(1+e^{-x})$ to produce a value in $[0,1]$. Here the output is left unbounded. The reason is explained below in the comparison with vanilla GANs.

The discriminator is distinct from the pretrained k-GNN classifier Φ used in $\mathcal{L}_{\text{GNN}}$: $\mathcal{D}$ is trained from scratch during GIN-Graph training and scores graphs on realism, while Φ is frozen and scores graphs on class membership.

3.4.2 $\mathcal{L}_{\text{GAN}}$: The Two-Player Setup

The generator $\mathcal{G}$ and the discriminator $\mathcal{D}$ are trained in alternating steps:

- The **discriminator** receives a batch of real graphs from the training set and a batch of generated graphs from the current generator, and updates its weights to increase scores on real graphs and decrease scores on generated graphs.
- The **generator** produces a batch of graphs, passes them through the discriminator, and updates its weights to increase the scores the discriminator assigns.

Neither network is trained to convergence; they co-evolve. In each training step, the discriminator updates once, then the generator updates once ($n_{\text{critic}} = 1$). This alternation is necessary. If $\mathcal{D}$ were trained to convergence against a fixed generator, its output would be correct (low on all generated graphs, high on all real graphs) but uninformative as a gradient signal: the generated region of input space would become flat, so small changes to a generated graph would not change $\mathcal{D}$'s score, and the generator would receive no direction for improvement. Keeping $\mathcal{D}$ partially trained ensures that its scores still vary across the region where the generator's current outputs lie.

3.4.3 $\mathcal{L}_D$: Discriminator Loss

With the setup in place, we can state the discriminator loss. In each training step, we draw a batch of $B = 64$ real graphs G from the training set and a batch of B fake graphs $\tilde{G}$ from the generator. The discriminator minimises:

$$\mathcal{L}_D = \underbrace{\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]}_{\text{score on fake}} - \underbrace{\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]}_{\text{score on real}} + \underbrace{\lambda_{\text{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]}_{\text{gradient penalty}}, \tag{19}$$

where each expectation $\mathbb{E}[\cdot]$ is in practice the average over one batch. Consider the first two terms:

- $\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]$ is the average score the discriminator assigns to generated graphs. The discriminator minimises $\mathcal{L}_D$, so it pushes this term down, assigning *low* scores to generated graphs.
- $-\mathbb{E}_{G\sim p_{\text{data}}}[\mathcal{D}(G)]$ is the negative of the average score on real graphs. Minimising this term pushes the average score on real graphs *up*.

Together, these two terms encode one instruction: push real scores up, push generated scores down. The third term, the gradient penalty, is a stability device explained below.

3.4.4 $\mathcal{L}_{\text{GAN}}$: Generator Loss

The generator's adversarial loss is the negation of the discriminator's "score on fake" term:

$$\mathcal{L}_{\text{GAN}} = -\mathbb{E}_{\tilde{G}\sim\mathcal{G}}[\mathcal{D}(\tilde{G})]. \tag{20}$$

The generator wants to maximise the score the discriminator assigns to its outputs. Since optimisers minimise by convention, we negate: minimising $-\mathbb{E}[\mathcal{D}(\tilde{G})]$ is equivalent to maximising $\mathbb{E}[\mathcal{D}(\tilde{G})]$.

Real graphs do not appear in $\mathcal{L}_{\text{GAN}}$. The generator never sees real data directly. It learns about the real distribution only through the gradient signal from $\mathcal{D}$: real graphs train $\mathcal{D}$, and $\mathcal{D}$'s gradient informs the generator how to adjust its outputs. This indirect chain, from real data to $\mathcal{D}$'s parameters to $\mathcal{D}$'s gradient to the generator's parameters, is the core mechanism of GANs.

3.4.5 $\mathcal{L}_{\text{GAN}}$: Why WGAN-GP and Not a Vanilla GAN

In a vanilla GAN, the discriminator ends in a sigmoid, so its output is a value in $[0, 1]$ interpreted as the probability of the input being real. The problem is gradient saturation. Once $\mathcal{D}$ can reliably separate real from generated graphs, the sigmoid output for generated graphs is pushed toward 0. The derivative of the sigmoid $\sigma(x) = 1/(1 + e^{-x})$ is $\sigma(x)(1 - \sigma(x))$, which approaches zero when $\sigma(x) \approx 0$ or $\sigma(x) \approx 1$. The generator's weight updates are computed via the chain rule as a product of derivatives, one of which is this sigmoid derivative. A near-zero factor makes the entire product near-zero, and the generator stops learning. This is the core difficulty of vanilla GAN training.

WGAN removes the sigmoid. The discriminator's output is an unbounded real number with no flat region. When $\mathcal{D}$ becomes strong, its output for generated graphs continues

decreasing, and the derivative remains non-degenerate, so the generator continues to receive a gradient signal.

Removing the sigmoid introduces a new problem: without any bound on $\mathcal{D}$'s output, it can reduce its loss without limit by scaling its weights up. Doubling all weights roughly doubles the output gap between real and generated scores. The gradient penalty in Equation (19) prevents this. The term

$$\lambda_{\mathrm{GP}}\,\mathbb{E}_{\hat{G}}\Big[(\|\nabla_{\hat{G}}\mathcal{D}(\hat{G})\|_2 - 1)^2\Big]$$

penalises deviation of the gradient magnitude of $\mathcal{D}$'s output (with respect to its input $\hat{G}$) from 1. Here $\hat{G} = \epsilon G + (1-\epsilon)\tilde{G}$ with $\epsilon \sim \mathrm{Uniform}(0,1)$ is a random interpolation between a real and a generated graph, so the penalty is evaluated at points between the two distributions. The coefficient $\lambda_{\mathrm{GP}} = 10$.

This constraint has two effects. First, $\mathcal{D}$ cannot reduce its loss by weight rescaling. Consider a linear discriminator $D_\psi(G) = \psi^\top \mathrm{vec}(G)$ as a tractable case. Then $\nabla_G D_\psi = \psi$, so $\|\nabla_G D_\psi\|_2 = \|\psi\|_2$. Rescaling the weights $\psi \mapsto k\psi$ rescales the gradient norm to $k\|\psi\|_2$, and the gradient-penalty term becomes $(k\|\psi\|_2 - 1)^2$, which grows quadratically in k and dominates $\mathcal{L}_D$ for large k. For the deep LeakyReLU discriminator used here, the activations are positively homogeneous ($\mathrm{LeakyReLU}_\alpha(c\,x) = c\,\mathrm{LeakyReLU}_\alpha(x)$ for $c \geq 0$), so scaling every weighted layer by k scales the output by k^L with L the number of weighted layers, and the gradient norm scales the same way. The penalty therefore forbids rescaling as a route to lower $\mathcal{L}_D$: any rescaling inflates the penalty much faster than it can inflate the adversarial gap between real and generated scores. Second, the gradient flowing from $\mathcal{D}$ into the generator remains at a bounded magnitude, neither vanishing (the vanilla GAN failure) nor exploding (the unconstrained WGAN failure).

In summary: a vanilla GAN's sigmoid saturates when $\mathcal{D}$ is confident, killing the generator's gradient. WGAN-GP removes the sigmoid and adds a gradient penalty that keeps the gradient magnitude near 1. The generator therefore receives a non-degenerate learning signal regardless of $\mathcal{D}$'s strength.

3.4.6 $\mathcal{L}_{\mathrm{GNN}}$: GNN Guidance Loss

The adversarial loss enforces structural realism but does not steer generation toward any particular class. The GNN guidance loss maximises the pretrained classifier's confidence on the target class c:

$$\mathcal{L}_{\mathrm{GNN}} = -\log p_\Phi(y = c \mid \tilde{G}), \tag{21}$$

where $p_\Phi(y = c \mid \tilde{G})$ is the softmax probability that the pretrained k-GNN Φ assigns to class c for the generated graph $\tilde{G}$. This is the negative log-likelihood of the target class. The classifier's weights are frozen; only the generator's weights are updated. Minimising $\mathcal{L}_{\mathrm{GNN}}$ pushes the generator to produce graphs that the classifier assigns to class c.

3.4.7 $\lambda(t)$: Dynamic Weighting

The balance parameter $\lambda(t)$ in Equation (18) follows a sigmoid schedule [Sun et al.(2025)]. The general form in Sun et al. is an affine map from a sigmoid output into an arbitrary range $[\lambda_{\min}, \lambda_{\max}]$; the full range $[0,1]$ is used here, so the floor and ceiling drop out and

the schedule reduces to

$$\lambda(t) = \sigma\left(k \cdot \left(\frac{2(t/T - p)}{1 - p} - 1\right)\right), \tag{22}$$

where T is the total number of training iterations ($T = 300$ epochs in our experiments), $p = 0.4$ is the fraction of training before λ begins increasing, $k = 10$ controls the steepness of the transition, and $\sigma(x) = 1/(1 + e^{-x})$ is the logistic sigmoid.

When $t \ll pT$ (the first $\sim$120 epochs), the sigmoid argument is strongly negative, giving $\lambda(t) \approx 0$: the total loss is $\mathcal{L}_G \approx \mathcal{L}_{\text{GAN}}$, and the generator focuses on producing structurally realistic graphs without class-specific pressure. As t passes pT, $\lambda(t)$ increases toward 1: the total loss shifts to $\mathcal{L}_G \approx \mathcal{L}_{\text{GNN}}$, directing the generator toward class-specific patterns. Without this schedule, the generator collapses to structurally degenerate graphs that satisfy the classifier but bear no resemblance to real data, since $\mathcal{L}_{\text{GNN}}$ can be minimised by any graph that triggers the correct class output, regardless of structural plausibility.

3.4.8 $\mathcal{L}_{\text{reg}}$: Structural Regularisation

The generator outputs a fixed-size $N \times N$ adjacency matrix and $N \times D$ feature matrix ($N = 28$ for MUTAG, $N = 50$ for PROTEINS). Without explicit constraints, the Gumbel-Softmax sampling activates most slots and settles on atom-type distributions that satisfy the classifier but do not resemble real class members. $\mathcal{L}_{\text{GAN}}$ constrains these statistics only indirectly through the discriminator, and the signal is too weak to prevent these failures. The regularisation term $\mathcal{L}_{\text{reg}}$ supplies direct supervision on two graph-level statistics: graph size and atom-type frequencies. Both targets are precomputed once from the real training set and remain fixed during training.

The regulariser decomposes as $\mathcal{L}_{\text{reg}} = \mathcal{L}_{\text{size}} + \mathcal{L}_{\text{type}}$, combined into the full expression:

$$\mathcal{L}_{\text{reg}} = \lambda_{\text{size}} \cdot \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c}\right)^2 + \lambda_{\text{type}} \cdot \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2, \tag{23}$$

where $\mathcal{L}_{\text{size}}$ penalises deviation from the class mean graph size and $\mathcal{L}_{\text{type}}$ penalises deviation from the class atom-type distribution. We now derive each term.

Size penalty. The generator outputs a fixed $N \times N$ matrix with no flag marking which slots are active. A slot is effectively active if it has at least one edge to another slot, so the natural notion of graph size is the number of slots with at least one edge.

Define the soft degree of slot v as the row-sum of the soft adjacency matrix:

$$d_v = \sum_{u=1}^{N} \tilde{A}_{vu}.$$

A slot with no edges has $d_v \approx 0$; a slot with one edge has $d_v \approx 1$; a slot with three edges has $d_v \approx 3$. The naive active-slot count would use an indicator:

$$\hat{n}_{\text{naive}} = \sum_{v=1}^{N} \mathbb{1}[d_v > 0.5],$$

treating a slot as active if it has at least half an edge of connection. This does not work for training: the indicator is a step function with derivative zero almost everywhere and undefined at the step. Any gradient flowing from $\mathcal{L}_{\text{size}}$ through this path collapses to zero.

We therefore replace the step with a smooth approximation. The logistic sigmoid $\sigma(x) = 1/(1+e^{-x})$ approximates a step at $x = 0$. To place the transition at $d_v = 0.5$ and sharpen it, we shift the argument by 0.5 and scale by 10:

$$\hat{n} = \sum_{v=1}^{N} \sigma\big(10 \cdot (d_v - 0.5)\big).$$

The multiplier 10 is steep enough that slots with $d_v < 0.3$ contribute near zero and slots with $d_v > 0.7$ contribute near one:

d_v	0	0.3	0.5	0.7	1.0
$\sigma(10(d_v - 0.5))$	0.007	0.12	0.50	0.88	0.99

The expression approximates the active-slot count on the forward pass while providing non-zero gradients everywhere on the backward pass.

With $\hat{n}$ (soft count of active slots) and $\bar{n}_c$ (mean node count of real class-c graphs, pre-computed once), the size penalty is

$$\mathcal{L}_{\text{size}} = \lambda_{\text{size}} \left(\frac{\hat{n} - \bar{n}_c}{\bar{n}_c} \right)^2 .$$

Two design choices matter. The squared error is smooth at zero with derivative vanishing as the error approaches zero, so the generator settles into the target without oscillation; the absolute value has a non-differentiable corner at zero and a constant-magnitude gradient. The division by $\bar{n}_c$ converts the error into a relative error, making the penalty scale-invariant across datasets: a 5-node error on MUTAG class 0 ($\bar{n}_c \approx 13.9$) is a $\sim$36% failure, while the same error on a dataset with $\bar{n}_c \approx 40$ is only $\sim$12.5%. Without this normalisation, λ_{size} would need to be retuned for every new dataset.

The coefficient $\lambda_{\text{size}} = 10$ is ten times larger than λ_{type} because a size violation breaks the graph structurally (a 50-node molecule where the class mean is 14 is unrecognisable), while atom-type violations are a softer failure.

Atom-type penalty. For each class c, the target distribution $p^c = (p^c_1, \ldots, p^c_D)$ is the frequency of each atom type in real class-c graphs, where p^c_i is the fraction of atoms of type i. This is a probability vector: entries in $[0, 1]$ summing to 1. For MUTAG class 0, $p^c \approx (0.85, 0.08, 0.04, \ldots)$ across the $D = 7$ atom types, with carbon dominating. Computed once, stored, reused.

The generated frequencies are computed from $\tilde{X}$. Each row of $\tilde{X}$ is nearly one-hot after Gumbel-Softmax, so summing column i counts how many slots selected atom type i:

$$\hat{p}_i = \frac{1}{N} \sum_{v=1}^{N} \tilde{X}_{v,i}.$$

Both $\hat{p}$ and p^c are probability distributions over the same D atom types. The penalty is the squared L2 distance between them, summed across types:

$$\mathcal{L}_{\text{type}} = \lambda_{\text{type}} \sum_{i=1}^{D} (\hat{p}_i - p_i^c)^2,$$

with $\lambda_{\text{type}} = 1$. The coefficient is smaller than λ_{size} because the sum already accumulates D squared errors and reaches meaningful magnitudes with a unit weight.

The natural distance between probability distributions is KL divergence, not L2. We use L2 for three reasons:

1. KL is undefined when $p_i^c = 0$, because the formula contains $\log(\hat{p}_i / p_i^c)$. On MUTAG, some classes contain zero instances of some atom types, so this is a concrete problem.
2. KL is asymmetric: $D_{\text{KL}}(\hat{p} \,\|\, p^c) \neq D_{\text{KL}}(p^c \,\|\, \hat{p})$. The two directions favour different failure modes and selecting one requires arbitrary justification. L2 is symmetric by construction.
3. KL has a gradient with a $1/p_i^c$ factor that explodes for rare atom types. The L2 gradient is $2(\hat{p}_i - p_i^c)$, bounded and stable.

Gradient path. $\mathcal{L}_{\text{reg}}$ has the shortest gradient path of the three loss components. Its gradients flow from scalar arithmetic on $(\tilde{A}, \tilde{X})$ through the Gumbel-Softmax backward pass into the generator MLP, without invoking the discriminator or the classifier. It is therefore the cheapest term per training step, since every other term requires a full network forward and backward pass while $\mathcal{L}_{\text{reg}}$ is closed-form. The target class enters only through the precomputed statistics $\bar{n}_c$ and p^c, so changing the target class requires no change to the computation graph.

3.5 Dense Wrapper for Differentiable k-GNN Inference

The pretrained classifier cannot be used directly inside the generator's training loop. This section explains why, and how we fix it. The 1-GNN dense wrapper is taken from GIN-Graph [Sun et al.(2025)]; the same logic is applied to the 2-GNN branch of the 1-2-GNN classifier in Section 3.5.4.

3.5.1 Why the sparse form breaks the gradient

The classifier was trained using PyTorch Geometric's sparse graph representation: the adjacency is stored as an `edge_index` tensor of integer source–destination pairs, and message passing is implemented as a `scatter_add` over edges rather than as a dense matrix product $\mathbf{AH}$. On discrete $\{0, 1\}$ inputs the two forms compute identical outputs; sparse is simply faster and more memory-efficient because it skips the $O(N^2)$ zero-multiplications that dominate $\mathbf{AH}$ when $\mathbf{A}$ is sparse.

This is fine for classifier training, where $\mathbf{A}$ is a fixed discrete input and no gradients need to flow through it. It breaks inside the generator's training loop, where the adjacency $\tilde{A} \in [0, 1]^{N \times N}$ is a continuous tensor produced by the Gumbel-Softmax straight-through estimator and $\nabla_\phi \mathcal{L}_{\text{GNN}}$ requires the derivative $\partial f_\Phi / \partial \tilde{A}$. The sparse forward has no slot for a float adjacency: converting $\tilde{A}$ into an `edge_index` requires a thresholding step whose

derivative is zero everywhere. The chain rule for $\nabla_\phi \mathcal{L}_{\text{GNN}}$ then contains a zero factor, and the generator receives no signal from this term.

3.5.2 The fix

The classifier's forward pass is re-implemented using dense matrix operations. The pretrained weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}, \mathbf{W}_3^{(\ell)}, \mathbf{W}_4^{(\ell)}$ are copied from the sparse model unchanged and kept frozen throughout GIN-Graph training. Where the sparse version uses `scatter_add`, the dense version uses `torch.matmul` with $\tilde{A}$ as a continuous input tensor. Autograd differentiates matrix multiplication: for $\mathbf{Y} = \tilde{A}\,\mathbf{H}$, each entry $\partial \mathbf{Y}/\partial \tilde{A}_{ij} = \mathbf{H}[j,:]$ is well-defined. The chain rule no longer contains a zero factor, and $\nabla_\phi \mathcal{L}_{\text{GNN}}$ flows back to the generator.

The wrapper is not a different model—it is the same mathematical function with a forward that accepts continuous adjacency matrices. The rewrite is direct for the 1-GNN (Section 3.5.3, following [Sun et al.(2025)]); the 1-2-GNN extension (Section 3.5.4) is this work's contribution and is treated separately.

3.5.3 Dense 1-GNN

For the 1-GNN branch, the rewrite is direct. The node-level update (Equation 4) translates to dense batched computation. For a batch of $B = 64$ graphs with node features $\mathbf{X} \in \mathbb{R}^{B\times N\times d}$ and adjacency $\mathbf{A} \in [0,1]^{B\times N\times N}$:

$$\mathbf{H}^{(t)} = \sigma\Big(\mathbf{H}^{(t-1)}\mathbf{W}_1^{(t)} + \mathbf{A}\,\mathbf{H}^{(t-1)}\mathbf{W}_2^{(t)}\Big), \tag{24}$$

where the batch dimension is broadcast. The matrix product $\mathbf{A}\,\mathbf{H}^{(t-1)}$ replaces `scatter_add`. Each entry $\tilde{A}_{ij}$ participates directly in the multiplication, so autograd can compute $\partial \mathbf{H}^{(t)}/\partial \tilde{A}_{ij}$. The weights $\mathbf{W}_1^{(t)}, \mathbf{W}_2^{(t)}$ are copied from the pretrained sparse model and kept frozen. This step is taken directly from GIN-Graph [Sun et al.(2025)] and similar dense rewrites for differentiable graph generation. It is stated here to contrast with the 2-GNN case.

3.5.4 Dense 2-GNN

The same sparse-to-dense logic is applied to the 2-GNN branch. The construction is more involved than for the 1-GNN because the 2-set neighbourhood depends on adjacency lookups, which must also be made continuous.

Recall from Section 3.2 that the 2-GNN operates on node pairs $\{i, j\}$, and the neighbourhood definition (Equation 8) requires checking whether a candidate node w is adjacent in the original graph: $A_{iw} = 1$ when replacing i, or $A_{jw} = 1$ when replacing j. In the sparse implementation, this adjacency check uses a CSR (compressed sparse row) lookup: given node i, iterate over its edge list to find all w with $(i, w) \in E$. This is fast for discrete graphs, but it is the same type of discrete operation that blocks gradients, since the neighbour set is a list of integer indices with no derivative connecting it to $\tilde{A}$.

In the dense rewrite, this binary check becomes a continuous weight. Instead of checking whether w is a neighbour of i (answer: 0 or 1), we use $\tilde{A}_{iw} \in [0,1]$ directly. Candidate

pair features $\mathbf{g}[j, w]$ are multiplied by $\tilde{A}_{iw}$, contributing to the aggregation in proportion to $\tilde{A}_{iw}$. When $\tilde{A}_{iw} \approx 0$, the contribution is zero; when $\tilde{A}_{iw} \approx 1$, it is full; intermediate values contribute proportionally. On discrete $\{0, 1\}$ inputs, this reproduces the original neighbourhood rule exactly.

First, we construct pair features from the 1-GNN node embeddings $\mathbf{H}^{(T_1)} \in \mathbb{R}^{B\times N\times d_T}$. For each pair (i, j), we concatenate the two node embeddings in canonical min–max order with the continuous adjacency value $\tilde{A}_{ij}$:

$$\mathbf{F}[i, j] = \left[\mathbf{h}_{\min(i,j)} \parallel \mathbf{h}_{\max(i,j)} \parallel A_{ij}\right] \in \mathbb{R}^{2d_T+1}, \tag{25}$$

A 2-set $\{i, j\}$ is unordered, but the pair tensor $\mathbf{F}$ is indexed by the ordered tuple (i, j), so each pair occupies two tensor slots: $\mathbf{F}[i, j]$ and $\mathbf{F}[j, i]$. Without canonicalisation, these two slots would hold the same node embeddings in opposite concatenation order, $\mathbf{h}_i \parallel \mathbf{h}_j$ versus $\mathbf{h}_j \parallel \mathbf{h}_i$, and the downstream learnable transformation $\mathbf{W}_4^{(\ell)}$ would map them to different outputs, breaking the unordered-pair semantics of Equation (8). Sorting the indices by min and max enforces $\mathbf{F}[i, j] = \mathbf{F}[j, i]$ by construction, while preserving both node embeddings as distinct blocks so that the pair-level MLP can still distinguish their roles. The third entry $\tilde{A}_{ij}$ is the isomorphism-type indicator; in the sparse model this was a hard bit $\mathbb{1}[(i, j) \in E] \in \{0, 1\}$, whereas here it is the continuous value from the generator. It is already symmetric because $\tilde{A}$ is symmetrised at the sampling stage (Equation (16)) and requires no further canonicalisation.

For pair-level message passing, recall from Equation (8) that each 2-set $\{i, j\}$ has two kinds of neighbours: pairs formed by replacing i with some w adjacent to i, and pairs formed by replacing j with some w adjacent to j. In the dense version, "adjacent to i" is the continuous weight $\tilde{A}_{iw}$. Using the transformed features $\mathbf{g} = \mathbf{F}\,\mathbf{W}_4^{(\ell)} \in \mathbb{R}^{B\times N\times N\times d_T}$, the aggregation for pair (i, j) sums over all candidate w, weighted by $\tilde{A}$:

$$\mathrm{agg}_1[i, j] = \sum_{\substack{w=1\\ w\neq j}}^{N} A_{iw} \cdot \mathbf{g}[j, w] \qquad \text{(replace } i \text{ with } w \text{ adjacent to } i\text{)}, \tag{26}$$

$$\mathrm{agg}_2[i, j] = \sum_{\substack{w=1\\ w\neq i}}^{N} A_{jw} \cdot \mathbf{g}[i, w] \qquad \text{(replace } j \text{ with } w \text{ adjacent to } j\text{)}. \tag{27}$$

The exclusions $w \neq j$ in agg_1 and $w \neq i$ in agg_2 prevent including the self-pair $\{j, j\}$ or $\{i, i\}$, which is not a valid 2-set. The self-loop exclusion ($A_{ii} = 0$) removes $w = i$ from agg_1 and $w = j$ from agg_2, but the terms $w = j$ in agg_1 and $w = i$ in agg_2 must be explicitly masked. We zero the diagonal of $\mathbf{g}$ (setting $\mathbf{g}[k, k] = \mathbf{0}$ for all k) before summation, which eliminates both boundary terms: $\mathbf{g}[j, j] = \mathbf{0}$ removes the $w = j$ term in agg_1, and $\mathbf{g}[i, i] = \mathbf{0}$ removes the $w = i$ term in agg_2.

These sums are computed via Einstein summation (`einsum`) over the masked $\mathbf{g}$. The full 2-GNN layer update is:

$$\mathbf{F}^{(\ell+1)} = \sigma\Big(\mathbf{F}^{(\ell)}\,\mathbf{W}_3^{(\ell)} + \mathrm{agg}_1^{(\ell)} + \mathrm{agg}_2^{(\ell)}\Big). \tag{28}$$

After $T_2 = 2$ layers, the 2-GNN readout sums over the upper-triangular entries (one entry per canonical pair):

$$\mathbf{h}_G^{(2)} = \sum_{i<j} \mathbf{F}^{(T_2)}[i, j]. \tag{29}$$

3.5.5 Summary

The dense wrapper preserves the exact mathematical behaviour of the pretrained classifier on discrete graphs while extending its domain to continuous adjacency matrices. It is an implementation change, not a model change: all pretrained weights are reused unchanged. For the 1-GNN branch, the change is replacing `scatter_add` with `matmul`, as in GIN-Graph [Sun et al.(2025)]. For the 2-GNN branch, the discrete adjacency check $A_{iw} \in \{0, 1\}$ is replaced by a continuous weight $\tilde{A}_{iw} \in [0, 1]$ that participates in the computation graph. Without this, the GIN-Graph training loop, which requires $\partial f_\Phi / \partial \tilde{A}$ to be well-defined, cannot be run against a 1-2-GNN classifier.

3.6 Validation Score

At evaluation time, both the generator and the classifier are frozen; no gradients are computed. We sample 100 graphs per class at temperature $\tau = 0.1$ (which produces sharper, more discrete outputs than the training temperature of $\tau = 1.0$) and assign each graph a score that quantifies its quality as an explanation. The score must be low if the graph fails on any of three axes (class identity, internal representation, or structural plausibility), so that success on one axis cannot compensate for failure on another.

The composite validation score [Sun et al.(2025)] is:

$$v = (s \cdot p \cdot d)^{1/3}, \tag{30}$$

where p measures the classifier's prediction, s measures where the graph sits in the classifier's internal embedding space, and d measures whether the graph has a realistic amount of connectivity.

The geometric mean enforces non-compensation between components. If a generated graph has $p = 0.95$, $s = 0.95$, but $d = 0.05$ (correct class, correct embedding region, wrong connectivity), the geometric mean gives $v \approx 0.36$, correctly indicating failure. An arithmetic mean would give $v \approx 0.65$, above the validity threshold, hiding the structural problem. We now define each component.

3.6.1 Prediction Probability (p)

The prediction probability is the frozen classifier's softmax output for the target class:

$$p = p_\Phi(y = c \mid \tilde{G}).$$

This is the same quantity the generator was trained to maximise via $\mathcal{L}_{\text{GNN}}$ (Equation 21). For a well-trained generator, p is close to 1 at evaluation time. This is expected: the generator spent hundreds of epochs optimising this quantity. The component p therefore functions as a sanity check that the generator converged, not as a discriminator between good and poor generators. A generator producing graphs with $p < 0.5$ did not converge.

3.6.2 Embedding Similarity (s)

For any graph G the classifier processes, the forward pass produces a single vector $\mathbf{h}_G$ representing the entire graph. In the 1-GNN, this vector is the sum pool of the final node

embeddings: $\mathbf{h}_G = \sum_{v \in V} \mathbf{h}_v^{(T_1)} \in \mathbb{R}^{d_T}$ (Equation 5). In the 1-2-GNN, it is the concatenation of the node-level and pair-level readouts: $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{2d_T}$ (Equation 10). One vector per graph, in the classifier's embedding space. Every forward pass on any graph produces exactly one such vector.

The *class centroid* $\boldsymbol{\mu}_c$ is the mean of $\mathbf{h}_G$ over all real training graphs in class c:

$$\boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G.$$

For MUTAG class 1 (Mutagen), $\mathcal{G}_c$ contains 125 molecules, so $\boldsymbol{\mu}_c$ is the average of 125 embedding vectors. The centroid sits in the same space as any individual $\mathbf{h}_G$ and marks the average location where real class-c graphs lie according to the classifier. It is computed once using the classifier's sparse implementation (real graphs are discrete, so sparse is appropriate), stored in the GIN-Graph checkpoint, and reused for every evaluation.

The embedding similarity is then the cosine similarity between the generated graph's embedding and this centroid:

$$s = \cos(\mathbf{h}_{\tilde{G}}, \boldsymbol{\mu}_c) = \frac{\mathbf{h}_{\tilde{G}} \cdot \boldsymbol{\mu}_c}{\|\mathbf{h}_{\tilde{G}}\| \, \|\boldsymbol{\mu}_c\|}, \quad \text{where } \boldsymbol{\mu}_c = \frac{1}{|\mathcal{G}_c|} \sum_{G \in \mathcal{G}_c} \mathbf{h}_G. \tag{31}$$

Cosine similarity measures the angle between two vectors, normalised so that identical direction gives $s = 1$ and orthogonal vectors give $s = 0$. A high s means the generated graph occupies the same region of embedding space as real class-c graphs. A low s means the classifier assigns the graph to class c but via an internal representation far from typical class members; the generator has found an input that triggers the correct output without resembling the real data in the classifier's internal representation.

Note on the computation of s. The GIN-Graph reference implementation [Sun et al.(2025)] uses the prediction probability p as a proxy for embedding similarity, setting $s = p$ and reducing the validation score to $v = (p^2 \cdot d)^{1/3}$. In this work, s is computed as the actual cosine similarity between $\mathbf{h}_{\tilde{G}}$ and $\boldsymbol{\mu}_c$, using the dense wrapper's `get_embedding()` method to extract the embedding from the frozen classifier for a continuous-valued generated graph. On PROTEINS Non-Enzyme, this distinction is visible in the results: both models achieve high p (0.919 and 0.998) but low s (0.356 and 0.151), a difference that would not appear under the $s = p$ proxy.

3.6.3 Degree Score (d)

The average degree of a graph is $\bar{d}_{\tilde{G}} = 2|\tilde{E}|/|\tilde{V}|$: total degree divided by node count, since each edge contributes 1 to each of its two endpoints. Let μ_d and σ_d be the mean and standard deviation of average degrees across real class-c graphs, both precomputed once from the training set. The degree score is:

$$d = \exp\left(-\frac{(\bar{d}_{\tilde{G}} - \mu_d)^2}{2\sigma_d^2}\right), \tag{32}$$

which is a Gaussian centred at the class mean μ_d with width σ_d. When $\bar{d}_{\tilde{G}} = \mu_d$, the score is $d = 1$. As the generated degree deviates from the mean, d decreases: one standard deviation off gives $d \approx 0.61$, two standard deviations give $d \approx 0.14$, three give $d \approx 0.01$.

Classes with high variance in graph sizes get a wider (more permissive) Gaussian than classes with uniform sizes.

The degree score provides a structural sanity check that s and p cannot provide. Both s and p operate in the classifier's learned embedding space. A generator could produce graphs with correct class identity and correct embedding location but with edge counts far from any real graph. The degree score catches this failure mode. It does not capture fine-grained structural properties (motif frequencies, degree distributions), but it reliably rejects graphs with wrong connectivity at the coarsest level.

3.6.4 Validity Threshold

A generated graph is considered **valid** if two conditions hold: (i) $v > 0.5$, and (ii) $\bar{d}_{\tilde{G}}$ falls within 3 standard deviations of μ_d. The degree score d therefore appears twice: once as a soft component inside v, and once as a hard gate via the 3σ condition. The hard gate catches graphs with abnormal connectivity regardless of their s and p values. A graph with $\bar{d}_{\tilde{G}}$ far outside the class range is rejected even if it scores well on the other two components. The validity rates reported in Section 5 are filtered on both conditions.

3.6.5 Granularity (κ)

We additionally report:

$$\kappa = 1 - \min\left(1,\ \frac{n_{\tilde{G}}}{\bar{n}_c}\right), \tag{33}$$

where $n_{\tilde{G}}$ is the number of active (non-isolated) nodes in the generated graph and $\bar{n}_c$ is the mean node count of real class-c graphs (the same $\bar{n}_c$ used in $\mathcal{L}_{\text{size}}$, Equation 23). When $n_{\tilde{G}} \geq \bar{n}_c$, the ratio saturates at 1 and $\kappa = 0$: the explanation is graph-sized. When $n_{\tilde{G}} \ll \bar{n}_c$, κ approaches 1: the explanation is a small substructure.

In these experiments, $\kappa \approx 0$ for all configurations. This is a structural property of GIN-Graph, not a failure. The generator outputs fixed-size $N \times N$ tensors and $\mathcal{L}_{\text{size}}$ pushes $\hat{n}$ toward $\bar{n}_c$, so explanations are graph-sized by construction. Model-level explainers answer "what does a typical class member look like?" rather than "which subgraph drives this prediction?"; the latter requires instance-level methods such as GNNExplainer. We report κ for completeness and as a diagnostic; this limitation is discussed further in Section 6.

3.6.6 Evaluation Protocol

Each configuration generates 100 graphs and reports: (i) the validity rate, the fraction passing both the $v > 0.5$ and the 3σ degree conditions; (ii) the mean validation score, averaged across all 100 graphs; (iii) the component decomposition s, p, d (Table 3), where patterns such as high p with low s on PROTEINS Non-Enzyme would not appear under the $s = p$ proxy; and (iv) the top-ranked explanations sorted by v, shown in the figures. All four quantities are forward-pass computations on frozen networks. No training or gradients are involved.

4 Experimental Setup

4.1 Datasets

We evaluate on two standard graph classification benchmarks (Table 1).

Table 1: Dataset statistics. GIN Gen is the maximum number of nodes used for explanation generation.

Dataset	Graphs	Node Feats	Max Nodes	GIN Gen	Task
MUTAG	188	7 (atom types)	28	28	Mutagenicity
PROTEINS	1113	3 (sec. str.)	620	50	Enzyme / non-enzyme

MUTAG [Debnath et al.(1991)] contains 188 molecular graphs labelled by mutagenic effect on *Salmonella typhimurium*. Nodes represent atoms (C, N, O, F, I, Cl, Br) and edges represent chemical bonds. The raw TU labels are $\{-1, +1\}$ with $+1$ denoting mutagenic compounds (125 graphs) and -1 non-mutagenic compounds (63 graphs). PyG's `TUDataset` remaps the labels by ascending sort, so in this work *Non-Mutagen* is class 0 (63 graphs) and *Mutagen* is class 1 (125 graphs).

PROTEINS [Borgwardt et al.(2005)] contains 1113 protein graphs where nodes are secondary structure elements (helix, sheet, coil/turn) connected if they are neighbours in the amino acid sequence or within a distance threshold in 3D space. Using the same ascending-sort remap, raw label 1 (enzymes, 663 graphs) maps to class 0 and raw label 2 (non-enzymes, 450 graphs) maps to class 1, so the classes are *Enzyme* (class 0) and *Non-Enzyme* (class 1). For GIN-Graph generation, we cap the graph size at 50 nodes (the median protein graph has $\sim$26 nodes), as the explanations highlight key substructures rather than reproducing entire large graphs.

4.2 Model Configuration

All k-GNN models use a hidden dimension of $d_T = 64$. The 1-GNN uses $T_1 = 3$ message-passing layers; the 1-2-GNN uses 3 layers for the 1-GNN component and $T_2 = 2$ layers for the 2-GNN component. Both use dropout of 0.5 and are trained with Adam (lr = 0.01) for 100 epochs with cross-entropy loss. Data is split 80/20 into train/test with a fixed seed of 42.

The GIN-Graph generator uses a latent dimension $d_z = 32$, hidden dimension $d_h = 128$, and is trained for 300 epochs with Adam (lr = 0.001) and batch size 64. WGAN-GP uses $\lambda_{\text{GP}} = 10$ and trains the discriminator once per generator update ($n_{\text{critic}} = 1$). The dynamic weighting uses $p = 0.4$ and $k = 10$ (Equation 22). The generation size is fixed at $N = 28$ for MUTAG and $N = 50$ for PROTEINS.

4.3 Evaluation Protocol

For each model-dataset-class combination, we generate 100 explanation graphs at evaluation temperature $\tau = 0.1$ and evaluate them using the validation score (Equation 30). We report:

- **Validity rate**: fraction of explanations passing degree and score thresholds;
- **Mean validation score**: averaged over all 100 generated graphs;
- **Component scores**: embedding similarity (s), prediction probability (p), degree score (d);
- **Granularity**: how fine-grained the explanations are.

5 Results

The goal of this section is to describe, without evaluative judgement, what each classifier has internalised as the class concept on each dataset. We do not have a ground-truth notion of a "correct" explanation, so claims are restricted to describing where the real data, the 1-GNN-generated data, and the 1-2-GNN-generated data differ, and where they coincide. We do not claim one architecture produces better explanations than the other.

Section 5.1 (Part A) first checks that all three components of the pipeline—the classifiers, the generators, and the validation score—are functioning before any comparison is interpreted. Section 5.2 (Part B) then performs the three-way qualitative comparison per (dataset, class) cell.

5.1 Part A: Pipeline sanity check

5.1.1 Classifier accuracy

Table 2 reports test-set accuracy for both classifiers on both datasets. On MUTAG, both achieve the same best test accuracy (89.5%); on PROTEINS, the 1-GNN reaches 79.8% and the 1-2-GNN 78.9%. Both classifiers therefore provide a usable target for GIN-Graph training. The two-point accuracy gap on PROTEINS is below the single-seed noise level discussed in Section 6.4, so we do not read it as evidence of architectural superiority.

Table 2: k-GNN classification results. Best test accuracy reported over all training epochs.

Dataset	Model	Params	Final Test Acc	Best Acc	Best Epoch
MUTAG	1-GNN	21,570	81.6%	89.5%	84
MUTAG	1-2-GNN	50,370	76.3%	89.5%	18
PROTEINS	1-GNN	21,058	75.8%	79.8%	120
PROTEINS	1-2-GNN	49,858	73.1%	78.9%	75

5.1.2 Generator training convergence

All four generators reached the 300-epoch horizon without divergence on both datasets.

5.1.3 Validation-score component sanity

Table 3 reports the mean validation score v and its components (s, p, d) averaged over 100 graphs from each of the eight generators. Two observations confirm the components are working as intended:

- Prediction probability p is high ($p \geq 0.56$ across all cells, $p \geq 0.92$ in seven of eight) confirming that each generator has learned to target its trained class.
- Embedding similarity s and degree score d lie strictly inside $(0, 1)$ in every cell—they are not collapsed to 0 or 1, so the metric is informative rather than saturated.

The spread of s across cells (from 0.151 to 0.987) is the signal the qualitative comparison will interpret in Part B.

Table 3: Validation-score components per generator (mean over 100 generated graphs).

Dataset	**Class**	**Model**	v	s	p	d
MUTAG	0 (Non-Mutagen)	1-GNN	0.320	0.700	1.000	0.210
MUTAG	0 (Non-Mutagen)	1-2-GNN	0.434	0.771	1.000	0.314
MUTAG	1 (Mut.)	1-GNN	0.681	0.935	1.000	0.490
MUTAG	1 (Mut.)	1-2-GNN	0.735	0.933	1.000	0.565
PROTEINS	0 (Enz.)	1-GNN	0.986	0.987	0.986	0.984
PROTEINS	0 (Enz.)	1-2-GNN	0.760	0.782	0.564	0.996
PROTEINS	1 (Non-Enzyme)	1-GNN	0.330	0.356	0.919	0.837
PROTEINS	1 (Non-Enzyme)	1-2-GNN	0.524	0.151	0.998	0.961

5.2 Part B: Qualitative three-way comparison

For each (dataset, class) cell, we describe what the real class-c graphs look like, then what the 1-GNN-generated and 1-2-GNN-generated class-c graphs look like under the same structural summaries. The statistics come from 500 generated graphs per configuration; no retraining is performed. Each per-dataset table below places the three sources side by side (or as adjacent rows) for every class.

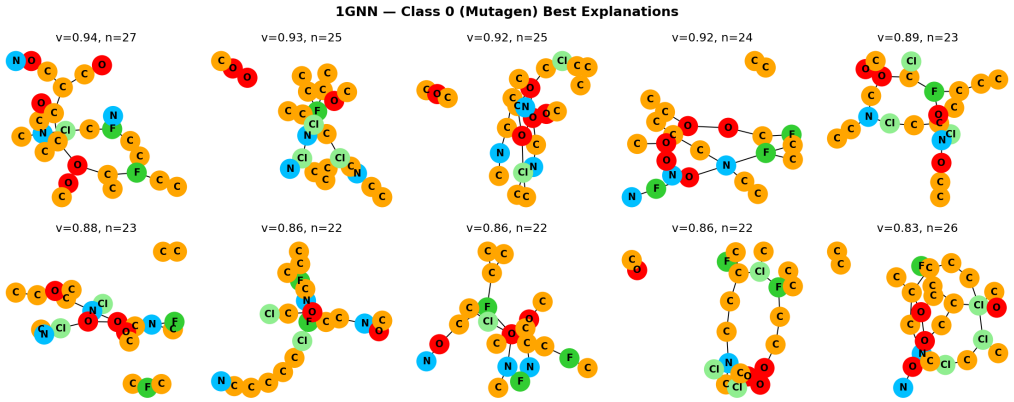
5.2.1 MUTAG

Table 4 summarises the three sources for MUTAG. The two generators both achieve $p \approx 1.0$ and keep mean degree within 0.05 of the real mean, but they diverge in atom-type composition and graph size.

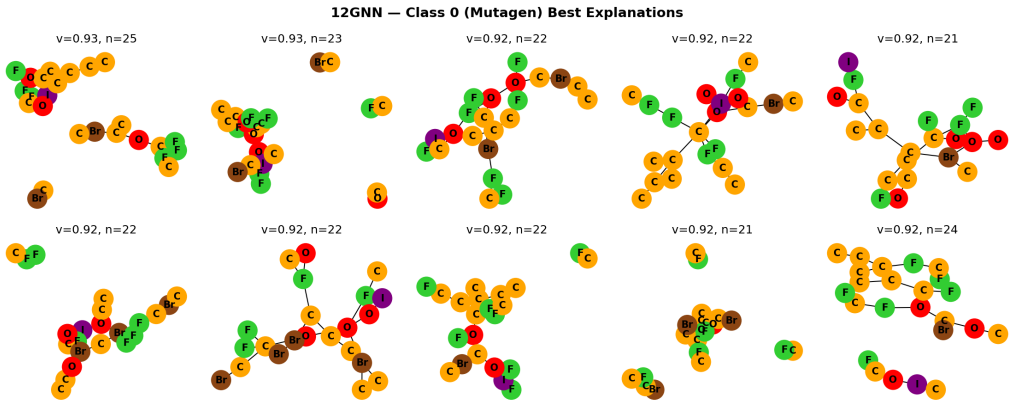
Table 4: MUTAG three-way comparison per class. Sizes and degree from 500 generated graphs. Atom percentages from generated-graph node populations; only the most populated atom types are shown.

Class	Source	$\bar{n}$	$\bar{d}$	%C	%N	%O	%halogen (F+Cl+Br+I)
0 (Non-Mutagen)	real	13.9	2.09	64.2	13.0	18.8	4.1
	1-GNN gen	23.8	2.06	57.2	10.9	17.0	14.9
	1-2-GNN gen	23.1	2.05	45.8	0.1	12.8	41.3
1 (Mutagen)	real	19.8	2.24	73.2	9.3	17.1	0.4
	1-GNN gen	27.3	2.23	63.4	7.6	19.3	9.7
	1-2-GNN gen	24.3	2.23	80.0	2.9	17.1	0.0

Figures 1 and 2 show top-ranked explanation graphs from both generators; Figure 3 gives the atom-type colour key.



(a) 1-GNN explanations for Non-Mutagen



(b) 1-2-GNN explanations for Non-Mutagen

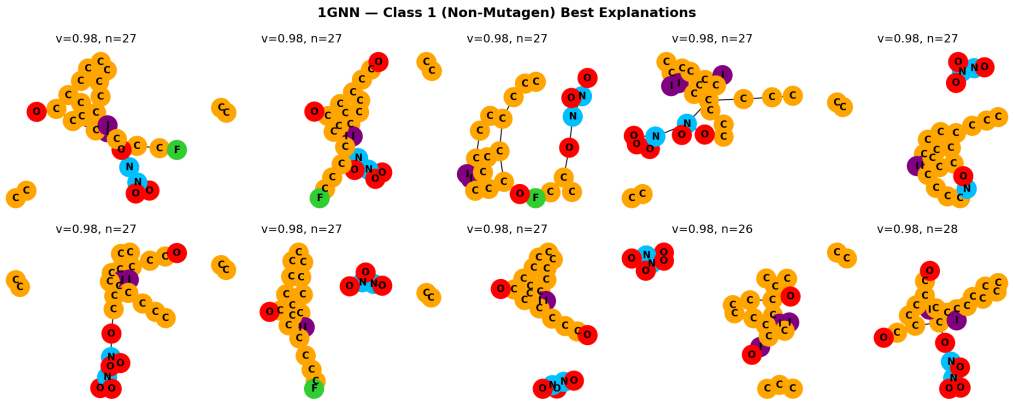
Figure 1: Top-ranked GIN-Graph explanations on MUTAG class 0 (Non-Mutagen). Node colours indicate atom types (see Figure 3).

Discussion (MUTAG). On class 0 (Non-Mutagen) the two architectures diverge *in different directions* from the real data: the 1-GNN keeps the real C/N/O balance roughly intact but amplifies halogens from 4.1% to 14.9%, while the 1-2-GNN drops nitrogen from 13.0% to 0.1% and amplifies halogens to 41.3%. On class 1 (Mutagen) the two generators diverge from the real data *in the same direction* in one respect—both produce oversized graphs ($\bar{n} \approx 25$ versus real 19.8)—but in different directions in another: the 1-GNN introduces iodine and fluorine that are nearly absent in the real class (0.4% halogen), while the 1-2-GNN produces essentially pure C/O/N backbones with no halogens. These are descriptive differences, not quality claims.

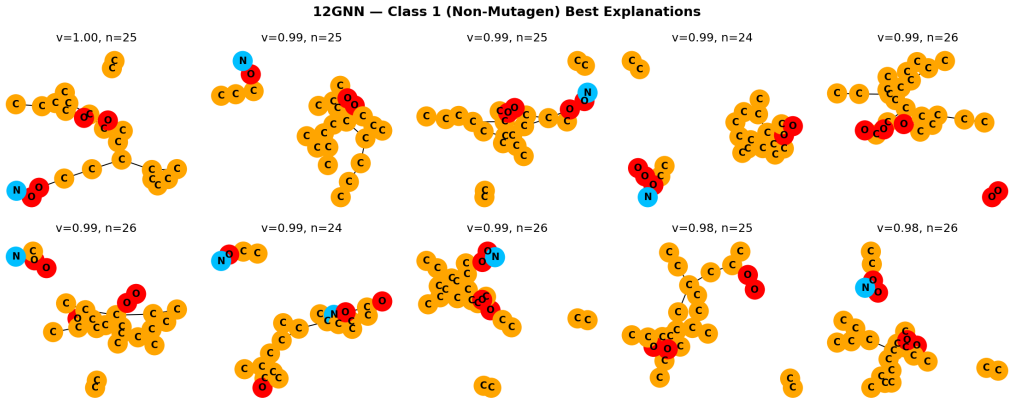
5.2.2 PROTEINS

Table 5 gives the analogous summary for PROTEINS. Here the node-type alphabet is secondary-structure labels (Helix, Sheet, Coil/Turn).

Figures 4 and 5 show top-ranked explanations for each class and model.



(a) 1-GNN explanations for Mutagen



(b) 1-2-GNN explanations for Mutagen

Figure 2: Top-ranked GIN-Graph explanations on MUTAG class 1 (Mutagen). Node colours indicate atom types (see Figure 3).

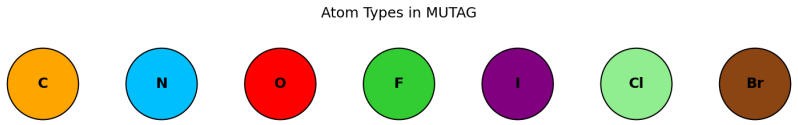
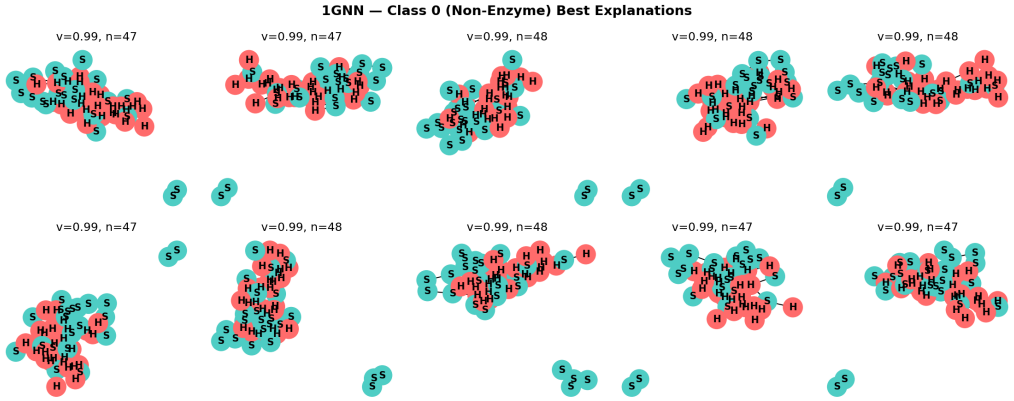


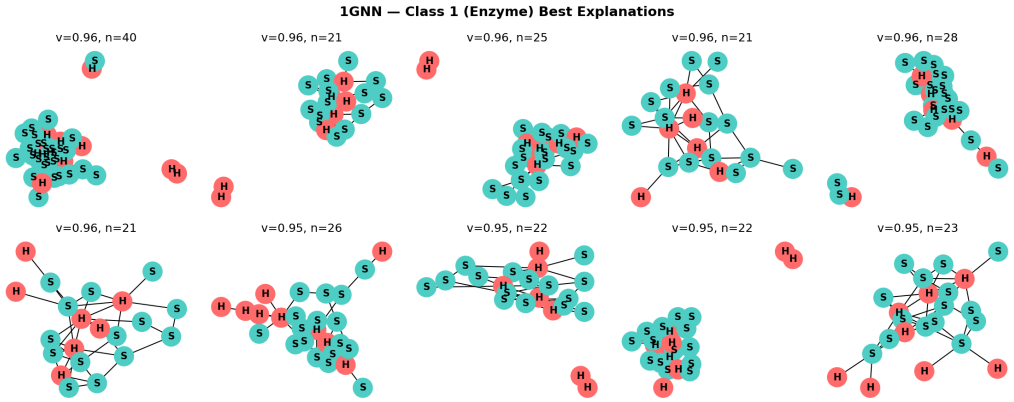
Figure 3: MUTAG atom type colour legend. Atoms: C (orange), N (blue), O (red), F (green), I (purple), Cl (light green), Br (brown).

Table 5: PROTEINS three-way comparison per class. Sizes and degree from 500 generated graphs. Node-type percentages from generated-graph node populations.

Class	Source	$\bar{n}$	$\bar{d}$	%Helix	%Sheet	%Coil/Turn
0 (Enzyme)	real	48.7	3.80	50.4	46.5	3.1
	1-GNN gen	46.6	3.81	54.3	45.7	0.0
	1-2-GNN gen	50.0	3.81	50.9	47.1	2.0
1 (Non-Enzyme)	real	23.7	3.64	40.7	56.2	3.1
	1-GNN gen	24.0	3.73	24.5	75.5	0.0
	1-2-GNN gen	29.5	3.64	11.0	88.9	0.0

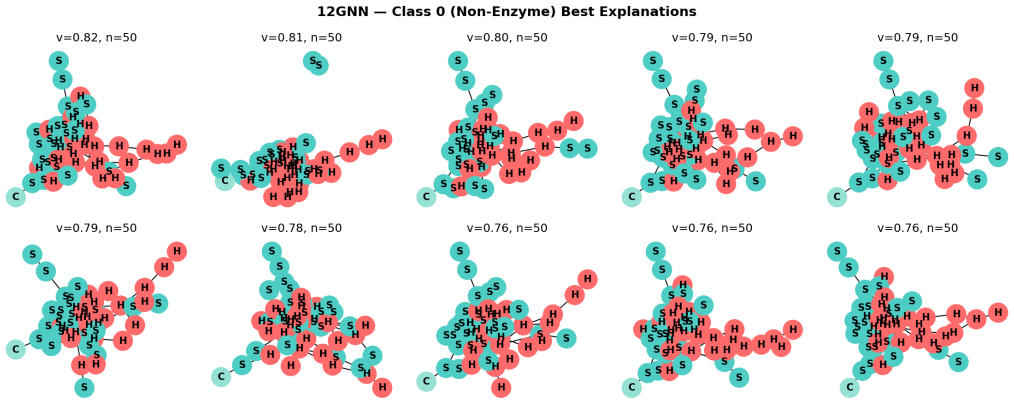


(a) 1-GNN explanations for Enzyme

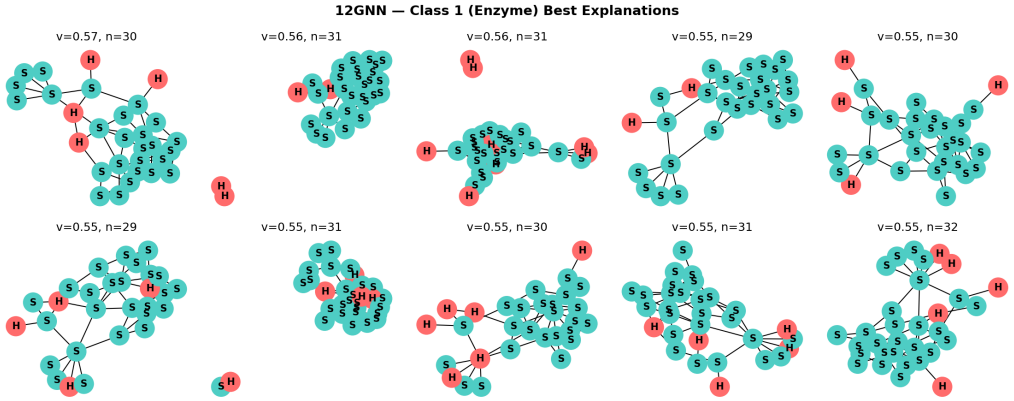


(b) 1-GNN explanations for Non-Enzyme

Figure 4: Top-ranked GIN-Graph explanations on PROTEINS (1-GNN). Node colours indicate secondary structure types: helix (H, pink), sheet (S, teal), coil/turn (C, green).



(a) 1-2-GNN explanations for Enzyme



(b) 1-2-GNN explanations for Non-Enzyme

Figure 5: Top-ranked GIN-Graph explanations on PROTEINS (1-2-GNN).

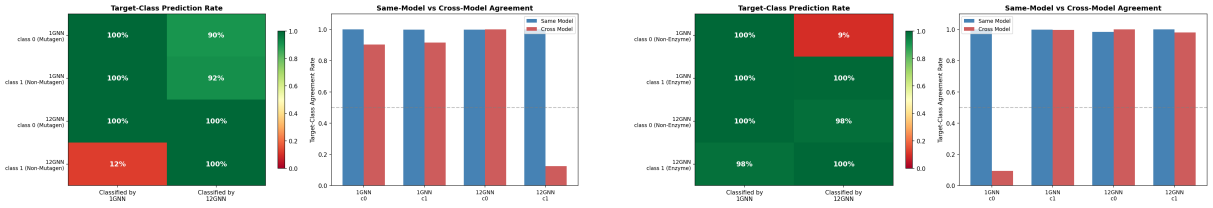
Discussion (PROTEINS). The headline observation on PROTEINS is not in the atom/structure-type columns but in the embedding similarity. Both classifiers assign high target-class probability to their own class-1 (Non-Enzyme) generations ($p = 0.919$ and $p = 0.998$, Table 3), yet the 1-GNN-generated graphs land at $s = 0.356$ in the 1-GNN embedding space while the 1-2-GNN-generated graphs land at $s = 0.151$ in the 1-2-GNN embedding space. Same confidence, different regions of each classifier's learned embedding geometry. This is the comparison's clearest indication that the two classifiers have internalised different Non-Enzyme concepts. On class 0 (Enzyme), the 1-2-GNN generator matches the real secondary-structure distribution closely (Helix 50.9% vs. real 50.4%, Sheet 47.1% vs. 46.5%), whereas the 1-GNN-generated size is slightly below the real mean (46.6 vs. 48.7). On class 1, both generators diverge from the real data in the same direction on graph size (oversized) but in different magnitudes on secondary-structure composition (the 1-2-GNN concentrates strongly on Sheet, 88.9% vs. real 56.2%). Again, these are descriptive observations; we are not claiming one generator is a more faithful sampler.

5.2.3 Cross-model classification

Table 6 and Figure 6 report what happens when each generated dataset is classified by *both* k-GNN classifiers. Same = target-class agreement by the generating model's own classifier; Cross = agreement by the other classifier.

Table 6: Cross-model classification of generated graphs (500 per class).

Generated by	Target class	Same	Cross
MUTAG			
1-GNN	Non-Mutagen (c0)	100%	90.4%
1-GNN	Mutagen (c1)	99.8%	91.6%
1-2-GNN	Non-Mutagen (c0)	99.8%	100%
1-2-GNN	Mutagen (c1)	100%	12.4%
PROTEINS			
1-GNN	Enzyme (c0)	100%	9.4%
1-GNN	Non-Enzyme (c1)	99.8%	99.6%
1-2-GNN	Enzyme (c0)	98.4%	100%
1-2-GNN	Non-Enzyme (c1)	100%	98.0%



(a) MUTAG (b) PROTEINS

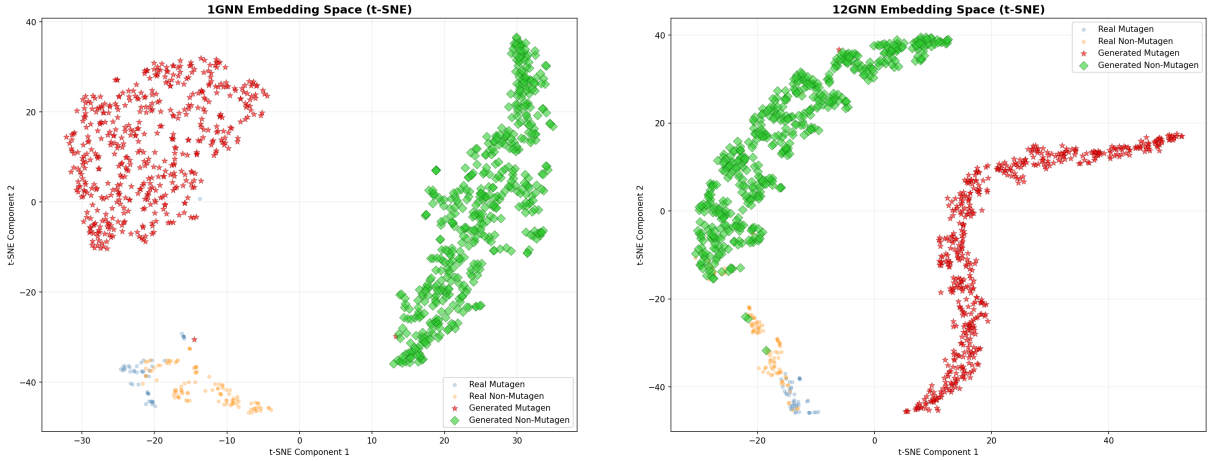
Figure 6: Cross-model classification of generated graphs. Each cell shows the fraction classified into the target class by each model.

Two cells stand out. The 1-2-GNN's Mutagen graphs on MUTAG are accepted as Mutagen by the 1-2-GNN (100%) but classified as Non-Mutagen by the 1-GNN (cross 12.4%).

Symmetrically, the 1-GNN's Enzyme graphs on PROTEINS are accepted by the 1-GNN (100%) but classified as Non-Enzyme by the 1-2-GNN (cross 9.4%). Each classifier has a class whose defining features, as captured by that classifier's generator, are not visible to the other classifier.

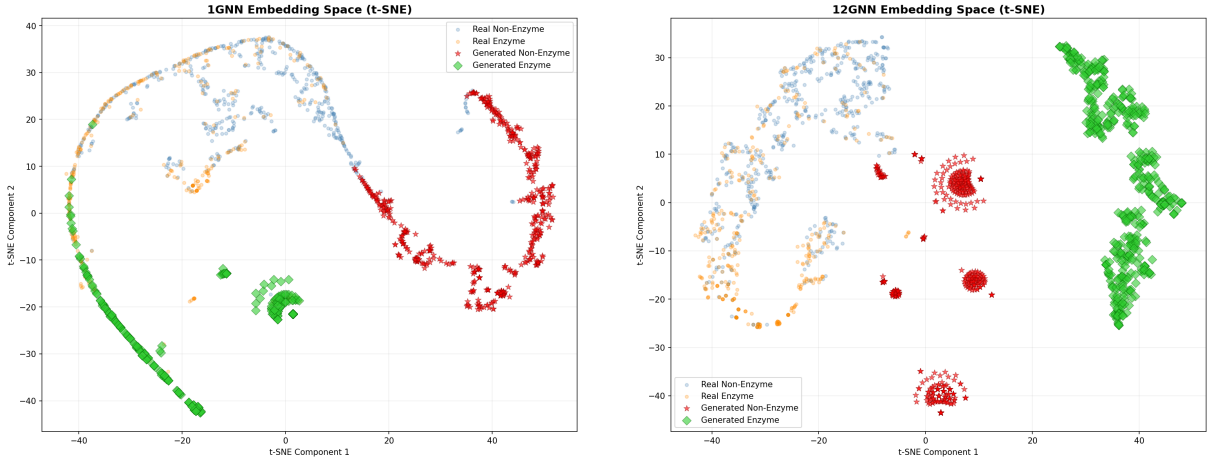
5.2.4 Embedding-space structure

Figures 7 and 8 show t-SNE projections of real and generated graph embeddings in each classifier's learned space.



(a) 1-GNN embedding space (b) 1-2-GNN embedding space

Figure 7: t-SNE projections of MUTAG real (small dots) and generated (large markers) graph embeddings. Colours distinguish Non-Mutagen (class 0) from Mutagen (class 1).



(a) 1-GNN embedding space (b) 1-2-GNN embedding space

Figure 8: t-SNE projections of PROTEINS real (small dots) and generated (large markers) graph embeddings. Colours distinguish Enzyme (class 0) from Non-Enzyme (class 1).

On MUTAG the generated graphs are well separated by class in each space but do not fill in the real-data clouds; they form their own class-consistent manifolds. On PROTEINS the effect is stronger: in the 1-2-GNN space the generated graphs collapse into tight disconnected prototype islands. Consistent with the high same-model agreement rates,

each generator samples a class-discriminative region of its target classifier's embedding space, but the class-level distribution of those samples does not coincide with the real data's distribution. Whether this is a property of the generator, the classifier, or both, the pipeline cannot distinguish.

5.3 Summary of the comparison

The three-way comparison shows:

- The 1-GNN and 1-2-GNN each induce a generator that produces high target-class probability ($p \geq 0.92$ in seven of eight cells) and local degree close to the real class mean.
- The node-type composition and graph size distributions of the generated data differ between the two architectures in every (dataset, class) cell, and each architecture diverges from the real data in a direction that is only partially shared with the other.
- Cross-model classification and embedding-space visualisations indicate that each classifier has class-defining features that are *not* recognised by the other classifier (12.4% and 9.4% agreement in the asymmetric cells).

This is evidence that the two architectures internalise different class concepts on these datasets. It is not evidence that one concept is closer to a ground-truth class definition; the pipeline provides no such ground truth. We also do not claim one architecture produces better explanations.

6 Discussion

6.1 Interpreting the observed divergence

The descriptive findings in Section 5.2 support a reading of the two architectures as internalising different class concepts. We emphasise that this is a statement about *what the classifiers encode*, not about which encoding is more correct: the pipeline provides no ground truth against which to judge. The cross-model classification results (12.4% on MUTAG Mutagen, 9.4% on PROTEINS Enzyme) are the strongest quantitative evidence for this reading, since they show that each classifier has class-defining features not visible to the other. The embedding t-SNE projections in Section 5.2 add a caveat: even within each classifier's own space, the generated graphs do not reproduce the real-data geometry, so the same-model agreement rates reflect the generator finding class-consistent regions rather than faithful sampling of the empirical distribution.

6.2 Local structure vs. population fidelity

The three-way comparison in Section 5.2 separates *local structural realism* from *population-level fidelity*: degree distributions are matched closely in every configuration, but graph-size and node-type distributions often are not. A generated graph's local connectivity can look plausible even when the overall population is systematically shifted. This distinction is the main reason we avoid reading any single summary number as a verdict on either architecture; fidelity is class- and configuration-specific, and neither architecture dominates

across cells.

6.3 The granularity problem

Most generated explanations have granularity $\kappa \approx 0$, meaning they remain close to full-graph prototypes rather than small substructures. This is a structural limitation of the GIN-Graph approach: the generator produces graphs of a fixed maximum size N, and the Gumbel-Softmax sampling tends to activate many nodes. For MUTAG ($N = 28$, average graph $\sim$18 nodes), the generated populations still average 23–27 active nodes. For PROTEINS ($N = 50$, average graph $\sim$26 nodes), the class 0 generators remain near the size cap (46.6 nodes for 1-GNN and 50.0 for 1-2-GNN), while the class 1 generators are somewhat smaller (24.0 and 29.5 nodes) but still do not behave like sparse subgraph explanations.

Model-level explanations are inherently coarse-grained: they answer "what does a typical class member look like?" rather than "which substructure drives the prediction?" This is a property of the method, not a flaw—the two questions require different explanation approaches.

6.4 Limitations

Several limitations qualify the findings:

Single seed, no error bars. All experiments use a single data split (seed 42) without cross-validation. With only 38 test graphs on MUTAG, the identical 89.5% best accuracy for both models could be coincidence—a difference of one misclassified graph changes accuracy by 2.6 percentage points. Similarly, the validity differences on MUTAG (82% vs. 78%, i.e. 4 graphs out of 100) may not be statistically significant without variance estimates across seeds, which is why we focus on the *qualitative* divergence in generated explanations rather than these marginal quantitative differences. The cross-model classification results (9.4%, 12.4% agreement) are extreme enough to be meaningful even without error bars. The PROTEINS results are more robust (224 test graphs), but the complementary class-specific pattern could also reflect data split artefacts.

No cross-validation. Best test accuracy is reported as the maximum over epochs (early stopping by oracle), which overestimates generalisation performance. A more rigorous evaluation would use a held-out validation set for model selection.

Classifier–generator coupling. The generator's ability to drive p toward 1 depends on the classifier's embedding geometry. The 1-2-GNN operates on a concatenated embedding $\mathbf{h}_G = [\mathbf{h}_G^{(1)} \| \mathbf{h}_G^{(2)}] \in \mathbb{R}^{128}$ with pair-level aggregation (Equations 26–27), which yields a more complex gradient landscape than the 1-GNN's $\mathbb{R}^{64}$. The lower $p = 0.564$ on PROTEINS Enzyme (versus the 1-GNN's $p = 0.986$) is consistent with this; none of the observed comparisons should be read as isolating the classifier from the generator.

Generator–classifier output space mismatch. A fundamental consideration is that GIN-Graph was designed for standard (1-WL) GNNs, and its generator architecture reflects this. The generator outputs an adjacency matrix $\tilde{A}$ and node feature matrix $\tilde{X}$—quantities that a 1-GNN directly consumes. A 1-GNN's decisions are based on node features and local neighbourhoods, which the generator can target directly: "place nitrogen

here, connect it to these carbons." The 1-2-GNN, however, makes decisions in a pairwise feature space derived from all $\binom{n}{2}$ node pairs. The generator has no mechanism to directly control pair-level representations—it can only influence them indirectly through edge and node choices. Despite this, the generator achieves high same-model agreement for all configurations ($\geq 94\%$) after structural regularisation, demonstrating that the indirect optimisation path is viable. However, the consistently lower prediction probability for the 1-2-GNN ($p = 0.564$ vs. 0.986 on PROTEINS class 0) suggests that a higher-order–aware generator—one that directly produces pair-level features—could improve results further.

Evaluation metrics. The validation score $v = (s \cdot p \cdot d)^{1/3}$ can be dominated by a single low component. The degree score, which measures structural realism via average degree alone, is a coarse proxy that does not capture finer structural properties like motif frequencies or degree distributions.

No baselines. We do not compare with instance-level explanation methods (GNNExplainer, PGExplainer) or other model-level approaches. GIN-Graph is the only model-level method we are aware of that generates full graphs via GAN, so direct model-level baselines are limited. However, comparing the *utility* of model-level vs. instance-level explanations on the same classifiers would contextualise our findings.

6.5 Future Work

A natural extension is a higher-order–aware generator. The output-space mismatch identified in Section 6.4 means the current generator influences pair-level representations only indirectly, through its edge and node choices. A generator that produces pair-level (or more generally k-set–level) features directly would remove this indirect path and allow a cleaner test of what a 1-2-GNN classifier has encoded, rather than what an unaware generator can coax out of it.

A second direction is extending the comparison to a 1-2-3-GNN, which adds 3-set message passing on top of node- and pair-level passes. This would test whether the kind of architecture-specific divergence observed here continues up the k-WL hierarchy, or whether it is a feature of the 1-to-2 step alone. The question is one of generality, not an expectation that a larger k produces better explanations.

7 Conclusion

We compared a standard 1-GNN with a hierarchical 1-2-GNN for model-level explanation generation using the GIN-Graph framework, with all configurations fully trained for 300 epochs across both datasets and both classes.

Our central finding is that the two architectures learn *qualitatively different internal representations* of graph classes. On MUTAG, both models achieve identical classification accuracy (89.5%), yet they generate strikingly different explanations: the 1-GNN produces more N/O-rich mutagenic structures, while the 1-2-GNN favours more strongly halogenated compounds. This divergence—same accuracy, different explanations—provides the cleanest evidence that higher-order message passing leads the model to encode different structural features, not merely "better" ones.

Cross-model classification of 500 generated graphs per class confirms this divergence quan-

titatively. On MUTAG, the 1-2-GNN's Mutagen graphs are rejected by the 1-GNN (12.4% agreement). On PROTEINS, 1-GNN-generated Enzyme graphs are rejected by the 1-2-GNN (9.4% agreement), while the 1-2-GNN's Enzyme graphs are accepted by both models (98.4%/100%). These asymmetries reveal architecture-specific decision boundaries, with the clearest selective advantage for the 1-2-GNN appearing on PROTEINS Enzyme and MUTAG Mutagen rather than uniformly across all settings.

At the same time, the generated-dataset comparison shows that the generators preserve local degree structure more reliably than graph-size distributions, and the embedding-space analysis shows that generated graphs often occupy separate prototype manifolds rather than the full real-data cloud. The most defensible conclusion is therefore nuanced: higher-order message passing changes what kinds of class prototypes are learned, and in selected cases improves fidelity, but it does not produce a blanket advantage on every class or every dataset.

Our fully differentiable dense wrapper enables GIN-Graph explanation generation for hierarchical k-GNN architectures while maintaining gradient flow through both node features and adjacency matrices. Additionally, our corrected validation metric—computing actual cosine similarity between generated and real graph embeddings rather than using prediction probability as a proxy—provides a more faithful measure of explanation quality. These technical contributions open the door to studying interpretability across the k-WL expressiveness hierarchy.

References

[Borgwardt et al.(2005)] K. M. Borgwardt, C. S. Ong, S. Schönauer, S. V. N. Vishwanathan, A. J. Smola, and H.-P. Kriegel. Protein function prediction via graph kernels. *Bioinformatics*, 21(suppl_1):i47–i56, 2005.

[Cai et al.(1992)] J.-Y. Cai, M. Fürer, and N. Immerman. An optimal lower bound on the number of variables for graph identification. *Combinatorica*, 12(4):389–410, 1992.

[Debnath et al.(1991)] A. K. Debnath, R. L. Lopez de Compadre, G. Debnath, A. J. Shusterman, and C. Hansch. Structure-activity relationship of mutagenic aromatic and heteroaromatic nitro compounds. *J. Med. Chem.*, 34(2):786–797, 1991.

[Gulrajani et al.(2017)] I. Gulrajani, F. Ahmed, M. Arjovsky, V. Dumoulin, and A. Courville. Improved training of Wasserstein GANs. In *NeurIPS*, 2017.

[Jang et al.(2017)] E. Jang, S. Gu, and B. Poole. Categorical reparameterization with Gumbel-softmax. In *ICLR*, 2017.

[Morris et al.(2019)] C. Morris, M. Ritzert, M. Fey, W. L. Hamilton, J. E. Lenssen, G. Rattan, and M. Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *AAAI*, 2019.

[Sun et al.(2025)] J. Sun, S. Pei, F. Zhang, and N. V. Chawla. GIN-Graph: A generative interpretation network for model-level explanation of graph neural networks. *Neurocomputing*, 2025.